LEARNING MANAGEMENT SYSTEM (LMS)
ANALYTICS USING HADOOP ECOSYSTEM

PROJECT DOCUMENTATION / TECHNICAL REPORT

Technology Stack: Hadoop • HDFS • Hive • HiveQL • Cloudera QuickStart VM • VirtualBox

Final Dataset: 10,000 LMS Activity Records
Approximately 1,000 Students • 3 Courses: C101, C102, C103

Student Name: A SHANKAR SAI SURYA CHANDRA
Registration No.: 2024019156
Class / Section: Big Data
Academic Year: 2024-2028

PROJECT TEAM MEMBERS

Registration Number	Name
2024001596	ADEPU HANSIKA
2024008031	N AESHAN CHANDRA REDDY
2024019156	A SHANKAR SAI SURYA CHANDRA
2024038240	SHREYA SARGAM

Table of Contents

1. Abstract

2. Introduction

3. Problem Statement

4. Motivation and Objectives

5. Scope

6. Technologies and Tools

7. System Architecture

8. Dataset Description

9. HDFS Implementation

10. Hive Implementation

11. Hive Partitioning

12. Data Validation

13. Analytics Methodology

14. Important HiveQL Queries

15. Results and Discussion

16. Engagement Classification

17. Risk Indicator

18. End-to-End Workflow

19. Advantages and Limitations

20. Future Scope

21. Conclusion

23. Project Screenshots / Execution Evidence

22. Demonstration Checklist

1. Abstract

This project demonstrates the use of the Hadoop ecosystem for Learning Management System (LMS) analytics. LMS platforms generate activity records containing information such as student identity, course, learning sessions, time spent, clicks, completion rate and assessment score. The project stores these records in HDFS and uses Apache Hive and HiveQL to perform scalable batch analytics.

The final project dataset contains 10,000 activity records representing approximately 1,000 students across three courses: C101, C102 and C103. A main Hive table named activity_logs_10k stores the activity data, while course_activity_10k is partitioned using course_id. Analytics include student engagement, course performance, completion, assessment performance, learning behavior and rule-based low-engagement indicators.

The observed course results show that C103 has the highest average score, 65.13, and the highest average completion, 60.57%, among the three courses.

2. Introduction

Educational platforms produce large volumes of digital learning traces. Individual activity records are useful but do not directly provide instructors with a clear view of engagement and performance. Big-data technologies can organize and process these records so that meaningful summaries can be produced.

This project applies Hadoop concepts to an LMS analytics use case. HDFS is used as the storage layer and Hive provides a structured query layer. The implementation was performed using the Cloudera QuickStart environment in VirtualBox.

3. Problem Statement

The project addresses the problem of converting raw LMS activity logs into useful academic insights. The system should allow an instructor to examine student engagement, course performance, completion patterns, learning behavior and possible low-engagement cases without manually processing thousands of rows.

4. Motivation and Objectives

- Demonstrate Hadoop-based educational data analytics.
- Store LMS activity data using HDFS.
- Analyze data using HiveQL rather than manually processing every record.
- Organize data through course-based Hive partitioning.
- Measure student engagement using time, clicks, completion and sessions.
- Compare course performance using score and completion.
- Create rule-based engagement and low-engagement indicators.

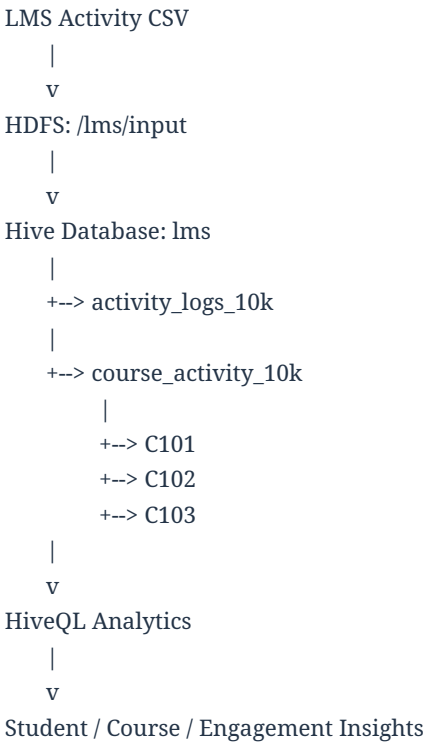
5. Scope

The implementation is a batch-oriented academic prototype. It covers CSV activity data, HDFS storage, Hive tables, HiveQL aggregation and course-based partitioning. It does not claim to implement a live institutional LMS, real-time streaming, production machine learning or a validated dropout prediction model.

6. Technologies and Tools

Technology	Purpose
Hadoop	Big-data ecosystem and distributed processing foundation
HDFS	Distributed storage for the activity dataset
Apache Hive	Data warehouse/query layer over Hadoop
HiveQL	SQL-like language for analytics
Cloudera QuickStart VM	Hadoop/Hive environment
VirtualBox	Virtual-machine platform
CSV	Input dataset format

7. System Architecture



HDFS provides distributed storage. Hive provides table management and SQL-like querying. Partitioning organizes the data by course and the HiveQL layer produces aggregated project outputs.

8. Dataset Description

Field	Type	Meaning
student_id	STRING	Unique learner identifier
course_id	STRING	Course identifier: C101, C102 or C103
session_id	STRING	Learning session/activity identifier
time_spent	INT	Time spent in the activity/session
clicks	INT	Number of LMS interactions
completion_rate	DOUBLE	Completion percentage
score	DOUBLE	Assessment score

Dataset summary:

Metric	Value
Activity records	10,000
Approximate students	1,000
Courses	3
Course IDs	C101, C102, C103
Attributes	7

9. HDFS Implementation

The dataset is stored in the HDFS input directory:

```
/lms/input
```

Useful verification commands are:

```
hdfs dfs -ls /lms/input
hdfs dfs -ls -h /lms/input
hdfs dfs -cat /lms/input/activity_logs_10000.csv | head
hdfs dfs -cat /lms/input/activity_logs_10000.csv | tail -n +2 | wc -l
```

These commands verify that the dataset exists in HDFS and that the expected number of data lines is present.

10. Hive Implementation

The project uses the Hive database named lms and the main table activity_logs_10k.

```
CREATE DATABASE IF NOT EXISTS lms;
USE lms;

CREATE TABLE activity_logs_10k (
   student_id STRING,
   course_id STRING,
   session_id STRING,
   time_spent INT,
   clicks INT,
   completion_rate DOUBLE,
   score DOUBLE
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
STORED AS TEXTFILE;
```

The input file can then be loaded from HDFS:

```
LOAD DATA INPATH '/lms/input/activity_logs_10000.csv' INTO TABLE activity_logs_10k;
```

Basic verification:

```
DESCRIBE activity_logs_10k;
SELECT COUNT(*) FROM activity_logs_10k;
SELECT * FROM activity_logs_10k LIMIT 10;
```

11. Hive Partitioning

The partitioned table course_activity_10k is organized using course_id.

```
CREATE TABLE course_activity_10k (
   student_id STRING,
   session_id STRING,
   time_spent INT,
   clicks INT,
   completion_rate DOUBLE,
   score DOUBLE
)
PARTITIONED BY (course_id STRING)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
STORED AS TEXTFILE;
```

Dynamic partitioning was enabled:

```
SET hive.exec.dynamic.partition=true;
SET hive.exec.dynamic.partition.mode=nonstrict;
```

The records are inserted into the appropriate course partition:

```
INSERT INTO TABLE course_activity_10k
PARTITION (course_id)
SELECT student_id, session_id, time_spent, clicks,
      completion_rate, score, course_id
FROM activity_logs_10k;
```

Verify the partitions with:

```
SHOW PARTITIONS course_activity_10k;
```

Expected partitions:

- course_id=C101
- course_id=C102
- course_id=C103

12. Data Validation

During development, the CSV header was found to have been interpreted as a data row. The project therefore filters the header value during analytical queries.

```
WHERE course_id IS NOT NULL
  AND course_id <> 'course_id'
```

This prevents the header from affecting averages, counts and other analytical results. In a production system, the input process should explicitly handle CSV headers before loading.

13. Analytics Methodology

Analysis	Measures	Purpose
Student engagement	Total time, clicks, completion, sessions	Understand learner activity
Course analytics	Students, sessions, time, clicks, completion, score	Compare courses
Student performance	Average score, completion and activity	Identify performance patterns
Course ranking	Average score and completion	Rank courses
Learning behavior	Sessions, time, clicks, completion, score	Study activity patterns
Engagement classification	Time and clicks	Create engagement groups
Low-engagement indicator	Time, clicks, completion	Flag cases for review

14. Important HiveQL Queries

14.1 Overall Dataset Statistics

```
SELECT
   COUNT(*) AS total_records,
   COUNT(DISTINCT student_id) AS total_students,
   COUNT(DISTINCT course_id) AS total_courses,
   COUNT(DISTINCT session_id) AS total_sessions
FROM activity_logs_10k
```

```
WHERE course_id IS NOT NULL
  AND course_id <> 'course_id';
```

14.2 Student Engagement

```
SELECT student_id,
    SUM(time_spent) AS total_time,
    SUM(clicks) AS total_clicks,
    ROUND(AVG(completion_rate),2) AS avg_completion,
    COUNT(DISTINCT session_id) AS sessions
FROM activity_logs_10k
WHERE course_id IS NOT NULL
  AND course_id <> 'course_id'
GROUP BY student_id;
```

14.3 Course Analytics

```
SELECT course_id,
    COUNT(DISTINCT student_id) AS students,
    COUNT(DISTINCT session_id) AS sessions,
    SUM(time_spent) AS total_time,
    SUM(clicks) AS total_clicks,
    ROUND(AVG(completion_rate),2) AS avg_completion,
    ROUND(AVG(score),2) AS avg_score
FROM activity_logs_10k
WHERE course_id IS NOT NULL
  AND course_id <> 'course_id'
GROUP BY course_id;
```

14.4 Course Performance

```
SELECT course_id,
    ROUND(AVG(score),2) AS avg_score,
    ROUND(AVG(completion_rate),2) AS avg_completion
FROM activity_logs_10k
WHERE course_id IS NOT NULL
  AND course_id <> 'course_id'
GROUP BY course_id;
```

14.5 Engagement Classification

```
SELECT student_id,
    SUM(time_spent) AS total_time,
    SUM(clicks) AS total_clicks,
    CASE
      WHEN SUM(time_spent) >= 1500
          AND SUM(clicks) >= 700 THEN 'Highly Engaged'
      WHEN SUM(time_spent) >= 800
          AND SUM(clicks) >= 400 THEN 'Moderately Engaged'
      ELSE 'Low Engagement'
    END AS engagement_level
FROM activity_logs_10k
WHERE course_id IS NOT NULL
  AND course_id <> 'course_id'
GROUP BY student_id;
```

14.6 Low-Engagement Indicator

```
SELECT student_id,
    SUM(time_spent) AS total_time,
    SUM(clicks) AS total_clicks,
    ROUND(AVG(completion_rate),2) AS avg_completion
FROM activity_logs_10k
WHERE course_id IS NOT NULL
  AND course_id <> 'course_id'
GROUP BY student_id
HAVING SUM(time_spent) < 500
   OR SUM(clicks) < 200
   OR AVG(completion_rate) < 50;
```

15. Results and Discussion

The main course comparison produced the following observed results:

Course	Average Score	Average Completion
C101	64.48	60.54%
C102	65.01	59.39%
C103	65.13	60.57%

- C103 has the highest average score: 65.13.
- C102 has an average score of 65.01.
- C101 has an average score of 64.48.
- C103 has the highest average completion: 60.57%.
- C102 has the lowest average completion: 59.39%.

These results are descriptive results from the project dataset. They should not be interpreted as statistically significant differences without further statistical testing.

16. Student Engagement Classification

Category	Rule
Highly Engaged	Time ≥ 1500 AND clicks ≥ 700
Moderately Engaged	Time ≥ 800 AND clicks ≥ 400
Low Engagement	Does not satisfy the above rules

These thresholds are project-defined rules used to demonstrate how aggregated LMS activity can be transformed into interpretable categories. They are not validated predictive thresholds.

17. Low-Engagement Risk Indicator

The project demonstrates a simple rule-based indicator for further review when a student has any of the following:

- Total time < 500
- Total clicks < 200
- Average completion < 50%

This is an analytical support indicator and not a machine-learning dropout prediction system.

18. End-to-End Workflow

1. Prepare LMS activity CSV
2. Upload activity_logs_10000.csv to HDFS
3. Verify /lms/input
4. Create/select Hive database lms
5. Create activity_logs_10k
6. Load the dataset
7. Validate schema, records and sample rows
8. Create course_activity_10k
9. Enable dynamic partitioning
10. Insert records into C101/C102/C103 partitions
11. Verify partitions
12. Execute HiveQL analytics
13. Compare course and student results
14. Present findings using tables/charts

19. Advantages and Limitations

Advantages

- Demonstrates distributed storage using HDFS.
- Provides SQL-like analysis through HiveQL.
- Supports aggregation of thousands of LMS records.
- Uses partitioning for course-wise organization.
- Produces multiple learner and course-level indicators.

Limitations

- Dataset is project-generated rather than live institutional data.
- Only three courses are represented.
- Current analysis is batch-oriented.
- Engagement thresholds are manually defined.
- Risk classification is not a validated ML model.
- Hive terminal is not a full graphical dashboard.

20. Future Scope

- Integrate a real LMS database or event source.
- Add more courses, semesters and departments.
- Use Apache Spark for advanced processing.
- Create Power BI or Python dashboards.
- Apply validated ML models for performance prediction.
- Develop and evaluate dropout-risk prediction.
- Add personalized content/course recommendations.
- Generate instructor alerts from meaningful activity patterns.

21. Conclusion

The LMS Analytics project demonstrates a complete Hadoop-based workflow for educational data analysis. The activity data is stored in HDFS, modeled in Hive, organized through course-based partitioning and analyzed using HiveQL. The resulting analytics provide measures of engagement, course performance, learning behavior, completion and assessment performance.

In the observed course results, C103 achieved the highest average score (65.13) and highest average completion (60.57%). The project provides a foundation that can be extended to larger datasets, real LMS integration, dashboards and validated machine-learning models.

22. Demonstration Checklist

Order	Live Demonstration	Purpose
1	hdfs dfs -ls /lms/input	Prove data is in HDFS
2	hive → USE lms;	Open project database
3	DESCRIBE activity_logs_10k;	Show schema
4	SELECT COUNT(*) ...	Show dataset size
5	SELECT * ... LIMIT 10;	Show sample records
6	SHOW PARTITIONS course_activity_10k;	Prove partitioning
7	Student engagement query	Show learner analytics
8	Course analytics query	Show course analytics
9	Course performance query	Show actual results
10	Explain findings	Connect outputs to project objectives

Key facts to remember: 10,000 records; approximately 1,000 students; three courses; three course partitions; C103 average score 65.13; C103 average completion 60.57%.

23. Project Screenshots / Execution Evidence

All embedded screenshots from the supplied Big Data Case Study PDF are included below as execution evidence. They are arranged in source order with captions for easy reference.

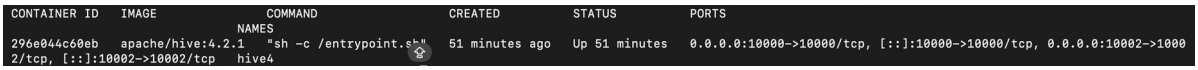


Figure 1. Docker / Hadoop environment verification output (source PDF, page 3).

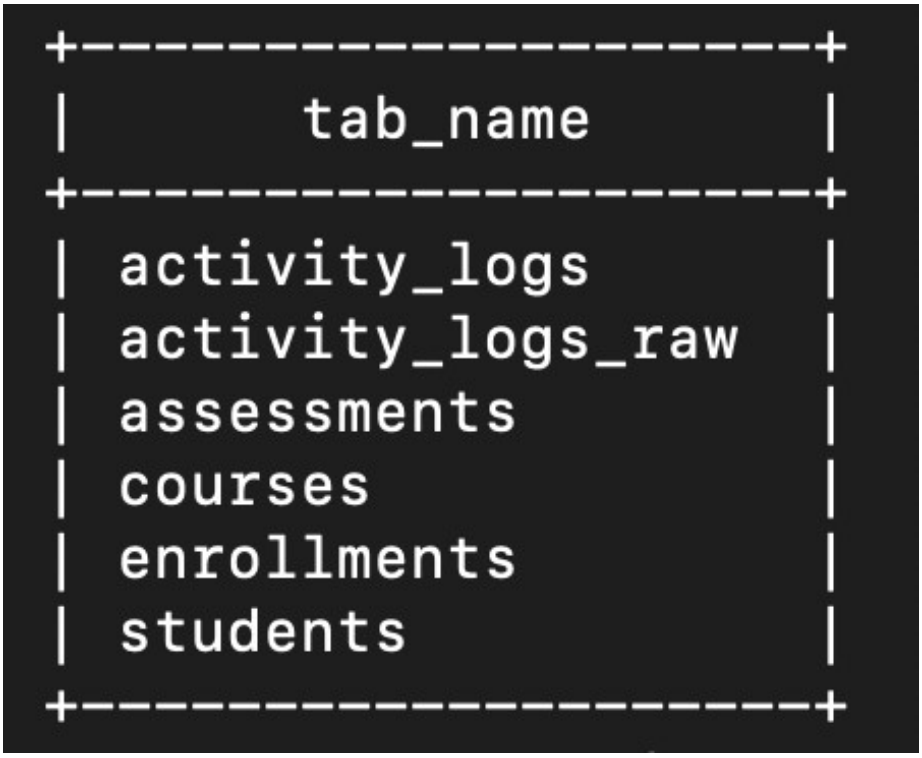
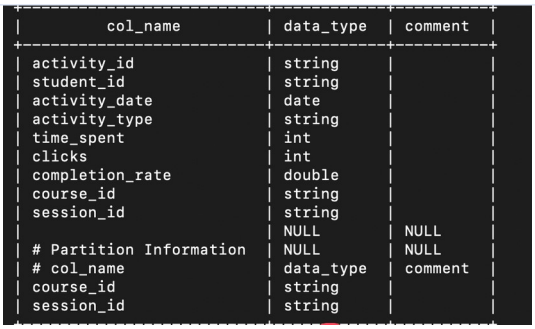
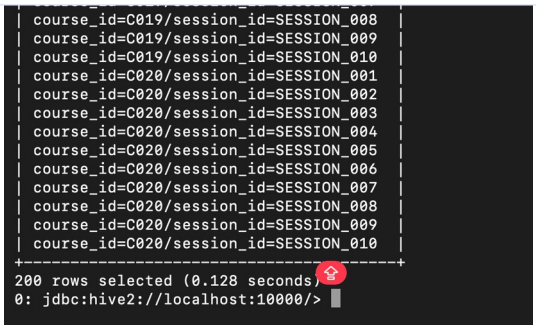


Figure 2. Hive table listing showing the LMS tables (source PDF, page 4).





Figures 3–4. Hive schema and partition verification outputs (source PDF, page 4).

student_id	avg_completion	avg_time	avg_clicks	dropout_risk
S0901	0.53	65.5	25.63	Low Risk
S0902	0.51	75.07	29.47	Low Risk
S0903	0.61	68.5	26.29	Low Risk
S0904	0.5	74.8	25.6	Low Risk
S0905	0.45	51.6	14.6	Low Risk
S0906	0.54	68.73	27.55	Low Risk
S0907	0.47	56.27	26.64	Low Risk
S0908	0.57	51.06	25.19	Low Risk
S0909	0.57	62.38	21.23	Low Risk
S0910	0.47	60.85	22.31	Low Risk
S0911	0.61	68.92	20.83	Low Risk
S0912	0.45	59.1	30.4	Low Risk
S0913	0.53	40.56	29.06	Medium Risk
S0914	0.49	79.88	25.0	Low Risk
S0915	0.65	62.71	18.71	Low Risk
S0916	0.41	63.0	26.78	Low Risk
S0917	0.44	73.0	28.17	Low Risk
S0918	0.46	48.25	39.75	Medium Risk
S0919	0.48	65.11	26.11	Low Risk
S0920	0.5	56.1	27.6	Low Risk
S0921	0.44	29.33	28.17	Medium Risk
S0922	0.48	55.25	23.13	Low Risk
S0923	0.43	70.5	30.58	Low Risk
S0924	0.43	51.78	29.0	Low Risk
S0925	0.64	55.63	30.69	Low Risk
S0926	0.51	82.0	31.25	Low Risk
S0927	0.55	51.67	24.17	Low Risk
S0928	0.41	52.33	22.87	Low Risk
S0929	0.44	66.33	33.67	Low Risk
S0930	0.72	57.23	27.62	Low Risk
S0931	0.66	84.63	27.38	Low Risk
S0932	0.62	71.54	21.38	Low Risk
S0933	0.63	50.33	19.22	Low Risk
S0934	0.56	77.0	23.33	Low Risk
S0935	0.4	48.5	22.6	Medium Risk
S0936	0.51	51.42	16.5	Low Risk
S0937	0.68	61.9	26.1	Low Risk
S0938	0.53	59.8	21.6	Low Risk
S0939	0.7	54.85	26.0	Low Risk
S0940	0.54	46.7	28.4	Medium Risk
S0941	0.6	62.4	24.5	Low Risk
S0942	0.46	35.88	19.5	Medium Risk
S0943	0.6	48.2	26.6	Medium Risk
S0944	0.44	62.07	29.73	Low Risk

Figure 5. Student dropout-risk classification output (source PDF, page 5).

c.course_id	c.course_name	avg_time_spent	avg_clicks	avg_completion_rate	activity_count
C009	Artificial Intelligence	61.4	24.97	0.56	506
C001	Python Programming	62.88	25.3	0.55	505
C002	Data Structures	60.62	25.24	0.54	502
C006	Big Data Analytics	58.41	25.07	0.54	496
C005	SQL Fundamentals	63.99	24.84	0.53	492
C003	Machine Learning	62.87	25.96	0.53	497
C011	Computer Networks	62.44	24.9	0.53	469
C014	Cyber Security	61.85	25.67	0.53	518
C004	Data Analytics	61.64	25.05	0.53	473
C019	Database Management	61.07	27.42	0.53	507

Figure 6. Course analytics result table (source PDF, page 6).

```
INFO  : Compiling command(queryId=hive_20260925100056_b2995a3a-826f-474f-a491-caa94063c16b): SELECT
    c.course_id,
    c.course_name,
    ROUND(AVG(a.time_spent),2) AS avg_time_spent,
    ROUND(AVG(a.clicks),2) AS avg_clicks,
    ROUND(AVG(a.completion_rate),2) AS avg_completion_rate,
    COUNT(*) AS activity_count
FROM activity_logs a
JOIN courses c
ON a.course_id = c.course_id
GROUP BY c.course_id, c.course_name
ORDER BY avg_completion_rate DESC,
         avg_time_spent DESC
LIMIT 10
```

Figure 7. HiveQL query used for course analytics (source PDF, page 6).

student_id	month	avg_score
S0996	2025-10	34.85
S0997	2025-02	64.92
S0997	2025-07	43.01
S0997	2025-08	92.27
S0997	2025-09	98.2
S0997	2025-10	64.6
S0997	2025-12	48.76
S0998	2025-02	66.59
S0998	2025-06	70.92
S0998	2025-08	83.64
S0998	2025-09	51.24
S0998	2025-10	85.47
S0998	2025-12	66.32
S0999	2025-08	85.79
S0999	2025-10	38.52
S0999	2025-11	30.57
S0999	2025-12	82.83

4,117 rows selected (0.811 seconds)
0: jdbc:hive2://localhost:10000/>

Figure 8. Monthly student performance result table (source PDF, page 6).

```
INFO  : Compiling command(queryId=hive_20260925100350_1f6072ec-e408-4daa-be54-f86fc3d2ae21): SELECT
    student_id,
    substr(CAST(assessment_date AS STRING),1,7) AS month,
    ROUND(AVG(score),2) AS avg_score
FROM assessments
GROUP BY student_id,
         substr(CAST(assessment_date AS STRING),1,7)
ORDER BY student_id, month
```

Figure 9. HiveQL query used for student performance trends (source PDF, page 6).

```
INFO  : Compiling command(queryId=hive_20260925100526_9140ece0-07aa-4103-8b13-28af67a66888): SELECT
    course_id,
    ROUND(AVG(time_spent),2) AS avg_time_spent,
    ROUND(AVG(clicks),2) AS avg_clicks,
    ROUND(AVG(completion_rate),2) AS avg_completion_rate,
    COUNT(*) AS activity_count
FROM activity_logs
GROUP BY course_id
ORDER BY avg_completion_rate DESC
```

Figure 10. HiveQL query used for behavioral analytics (source PDF, page 7).

course_id	avg_time_spent	avg_clicks	avg_completion_rate	activity_count
C009	61.4	24.97	0.56	506
C001	62.88	25.3	0.55	505
C002	60.62	25.24	0.54	502
C006	58.41	25.07	0.54	496
C011	62.44	24.9	0.53	469
C003	62.87	25.96	0.53	497
C004	61.64	25.05	0.53	473
C005	63.99	24.84	0.53	492
C012	60.81	25.61	0.53	519
C013	59.89	24.8	0.53	514
C014	61.85	25.67	0.53	518
C017	59.64	25.01	0.53	514
C019	61.07	27.42	0.53	507
C018	60.79	26.39	0.52	463
C008	62.0	25.8	0.52	504
C010	59.0	24.36	0.52	489
C016	61.3	25.55	0.52	471
C020	60.94	25.29	0.52	533
C015	60.81	25.04	0.51	510
C007	61.04	25.07	0.51	518

Figure 11. Behavioral analytics result table (source PDF, page 7).

End of Project Documentation and Execution Evidence